A Theory of Composable Lingos for Protocol Dialects

Víctor García[1], Santiago Escobar[1], Catherine Meadows[2], and Jose Meseguer[3]

[1] VRAIN, Universitat Politècnica de València, Valencia, Spain
{vicgarval,sescobar}@upv.es
[2] cameadows3306@gmail.com
[3] University of Illinois at Urbana-Champaign, Illinois, USA
meseguer@illinois.edu

Abstract. Formal patterns are formally specified solutions to frequently occurring distributed system problems that are generic, executable, and come with strong qualitative and/or quantitative formal guarantees. A formal pattern is a *generic system transformation* which transforms a usually infinite class of systems in need of the pattern's solution into enhanced versions of such systems that solve the problem in question. In this paper we demonstrate the application of formal patterns to *protocol dialects*. Dialects are methods for hardening protocols so as to endow them with light-weight security, especially against easy attacks that can lead to more serious ones. A *lingo* is a dialect's key security component, because attackers are unable to "speak" the lingo. A lingo's "talk" changes all the time, becoming a moving target for attackers. In this paper we present several formal patterns for both lingos and dialects. Lingo formal patterns can make lingos stronger by both *transforming* them and by *composing* several lingos into a stronger lingo. Dialects themselves can be obtained by the application of a single *dialect formal pattern*, generic on both the chosen lingo and the chosen protocol.

Keywords: Formal Patterns, Lingos, Compositions, Security, Maude

1 Introduction

Formal patterns [18] are methods for formally specifying generic solutions to distributed system problems that are executable and provide formal guarantees. Such patterns allow one to retain what has been learned about solving such problems without tying that knowledge to a particular concrete solution. They also allow for the composition of solutions and the composition of guarantees.

Formal patterns have been successfully applied to a number of different distributed systems problems [6, 19, 25]. They have also been applied to security protocols, and have been helpful in developing methods for hardening protocols against malicious attacks [4,7,21]. In this paper we apply this methodology to an emerging protocol hardening technique, the design of *protocol dialects*. Protocol dialects are methods for modifying protocols in order to provide a light-weight

layer of security, especially against relatively easy attacks that could potentially be leveraged into more serious ones. The scenario is usually that of a network of mutually trusting principals, e.g., an enterprise network, that needs to defend itself from outside attackers. The most effective approach is to prevent outside parties from even initiating a communication with group members, e.g., by requiring messages to be modified in some way unpredictable by the attacker.[4]

Protocol dialects are intended to be used as a first line of defense, not as a replacement for traditional authentication protocols. Indeed, they are primarily concerned with protecting communication between members of an enclave in which all parties trust each other against weak attackers that are trying to leverage off protocol vulnerabilities. We can make use of this restricted scope to use an attacker model based on the on-path attacker model [8,11]: First, the attacker can read messages, but cannot destroy or replace them. Second, principals trust each other and share a common secret that they can use, for example, as the seed for a pseudo-random number generator to generate and share common secret parameters. Because of the simplified threat model and trust assumption, dialects can be designed to be simpler than full-scale authentication protocols. Thus they can be expected to be prone to fewer implementation and configuration errors. With this in mind, dialects can be used as a separate but very thin layer in the protocol stack, useful as a defense against attacks in case the main authentication protocol is misconfigured or implemented incorrectly. Considering the dialect as a separate layer makes it easier to keep it simple, because it makes it possible to avoid dependencies on the layers it is communicating with. In our work, we consider dialects that sit right above the transport layer in the TCP/IP model, while the protocol it is modifying sits just above the dialect in the application layer. However, other locations are possible for the protocol being modified. Variations in the threat model are also possible, e.g., deciding whether or not to include replay or session hijacking attacks, which can both be implemented by an on-path attacker.

Protocol dialects as formal patterns were first introduced in [8]. A dialect is generic on two parameters: the given *lingo*, which describes the actual *message transformation*, and the underlying *protocol* whose security it improves. The *dialect* pattern is a *protocol transformation* yielding a new protocol responsible for applying the lingo and interfacing with the underlying protocol. Lingos can be transformed and composed into stronger ones in a compositional way. Using lingo compositions a given protocol can be made more secure in a wide variety of ways by means of the different dialects used in such lingo compositions.

We substantially extend the work in [8] by concentrating on the design of lingos, instead of the design of dialects as a whole, and developing *new formal patterns* to create and transform new lingos from simpler ones. This is a more flexible and computationally more efficient approach that increases the security of transformed lingos (and therefore of dialects) and provides a simpler methodology for designing new lingos and dialects out of previous ones and increasing

[4] By "message" we mean a unit of information transferred by the protocol, and can mean anything from a packet to a message in the traditional sense.

their security. In particular, we are able to *replace* two important formal patterns for dialects in [8] by much simpler similar formal patterns for lingos. Lingos rely on constantly changing secret *parameters*, which are generated by a pseudorandom number generator shared throughout, itself parametric on a single shared secret, an approach to lingo design introduced by Gogeneni et al. in [11].

From the software engineering point of view, the importance of formal patterns —in particular that of lingos and dialects— is that they make possible the design and verification of distributed systems of much higher quality than those developed in a traditional way, and they endow such systems with desired new features —for dialects with new lightweight security features— in a fully modular way. As explained in [20], formal patterns provide a modular, generic way of developing formal executable specifications of system *designs* that can be subjected to systematic formal verification *before* they are implemented and support the correct by construction development of high quality systems with a notable economy of effort and a high degree of reusability. Mathematically, the *dialect formal pattern* is a *theory transformation* definable as a partial function:

$$\mathcal{D} : \mathbf{PLingos} \times \mathbf{Protocols} \ni (\Lambda_p, \mathcal{P}) \mapsto \mathcal{D}_{\Lambda_p}(\mathcal{P}) \in \mathbf{Protocols}$$

where **PLingos** and **Protocols** denote classes of executable specifications, i.e., of executable *theories*, respectively specifying the classes of parametric lingos and communication protocols. Likewise, as we explain in the paper, *lingo formal patterns* are similar theory transformations mapping one or more lingos to a new lingo enjoying specific formal properties.

1.1 Related Work

Probably, the first work on dialects is that of Sjoholmsierchio et al. in [24]. In this work, the dialect is proposed as a variation of an open-source protocol to introduce new security measures or remove unused code, and is applied to the OpenFlow protocol, introducing, among other things a defense against cipher-suite downgrade attacks on TLS 1.2. This variation was independent of TLS, and was achieved by adding proxies, thus allowing for modification without touching the rest of the system and foreseeing the use of dialects as thin layers. In later work by Lukaszewski and Xie [14] a layer 4.5 in the TCP/IP model was proposed for dialects. In our own model we also make use of proxies for both sending and receiving parties, but stop short of proposing an additional official layer.

The threat model we use in this paper, and the one used in [8], is similar to that used by Goginieni et al. in [11] and Ren et. al [22]: an on-path attacker who is not able to corrupt any members of the enclave, as discussed earlier. We have also followed the suggestion in [22] to view dialects as composable protocol transformations. In this paper, we simplify this approach by reducing dialect transformations to more basic lingo transformations, which are generic, compositional methods for producing new lingos (and thus new dialects) from old ones.

In [10,11,15] Mei, Gogineni, et al. introduce an approach to dialects in which the message transformation is updated each time using a shared pseudo-random

number generator. This means that the security of a transformation depends as much, and possibly more, on its unpredictability as it does on the inability of the attacker to reproduce a particular instance of a transformation. We have adopted and extended this approach. In particular, we model a transformation as a parametrized function, or *lingo*, that produces a new message using the original message and the parameter as input. Both the lingo and the parameter can be chosen pseudo-randomly, as described in [8].

None of the work cited above (except [8]) applies formal design and evaluation techniques. However, in [26] Talcott applies formal techniques to the study of dialects running over unreliable transport, such as UDP. [26] is complementary to our work in a number of ways. First, we have been looking at dialects running over reliable transport, such as TCP. Secondly, we concentrate on a particular attacker model, the on-path attacker with no ability to compromise keys or corrupt participants, and then explore ways of generating dialects and lingos that can be used to defend against this type of attacker. In [26], however, the dialects are relatively simple, but the attacker models are explored in more detail, giving the attacker a set of specific actions that it can perform. Like our model, the work in [26] is formalized in the Maude formal specification language, providing a clear potential synergy between the two models in the future.

1.2 Contributions of this paper

1. We present a theory of lingos that adds important new *properties*, *transformations* and *composition operations* to the notion of lingo introduced in [8]. The main goal of this theory is to improve the security of protocol dialects.
2. We show that these new lingo transformations and composition operations are *formal patterns* [19] that guarantee desired properties and replace and greatly simplify two earlier formal patterns for dialects in [8]. These lingo transformations provide new compositional techniques for creating new lingos and dialects.
3. We provide formal executable specifications in Maude [5] for various lingos, their transformations and compositions, and dialects. For more information on them, see [9].

Section 2 substantially extends lingos beyond [8] and provides a wealth of new results about lingos and lingo transformations. Section 3 provides new lingo composition operations and proves their properties. Section 4 recalls the dialect notion and illustrates it with various examples. Section 5 concludes the paper.

2 Lingos

A lingo is a data transformation f between two data types D_1 and D_2 with a one-sided inverse transformation g. The transformation f is *parametric* on a secret parameter value a belonging to a parameter set A. For each parameter a, data from D_1 is transformed into data of D_2, which, using the same parameter

a, can be transformed back into the original data from D_1 using the one-sided inverse g. In all our applications, lingos will be used to transform the payload of a message of sort D_1 appearing in some protocol $\mathcal{P}$ whereas D_2 will then be the data type of transformed payloads. Such transformed payloads can then be sent, either as a single message or as a sequence of messages, to make it hard for malicious attackers to interfere with the communication of honest participants, who are the only ones knowing the current parameter $a \in A$. The point of a lingo is that when such transformed messages are received by an honest participant they can be transformed back using g (if they were broken into several packets they should first be reassembled) to get the original payload in D_1.

Briefly, the set D_1 can be thought of as analogous to the plaintext space, the set D_2 as analogous to the ciphertext space in cryptographic systems, and the parameter set A (also known as the secret parameter set) as analogous to the key space. The elements of A are generally required to be pseudorandomly generated from a secret shared by enclave members, so it can't be guessed in advance, and it is also generally updated with each use, so information gleaned from seeing one dialected message should not provide any help in breaking another. However, it is not necessarily required that an attacker is not be able to guess a after seeing $f(d_1, a)$. The only requirement is that it is not feasible for the attacker to compute $f(d_1, a)$ without having been told that a is the current parameter for $f(d_1, a)$. This is for two reasons. First, the lingo is intended to provide authentication, not secrecy. Secondly, the lingo is only intended to be secure against an on-path attacker [8] that can eavesdrop on traffic but not interfere with it. Thus, if an enclave member sends a message $f(d_1, a)$, the attacker, even if it learns a, can't remove $f(d_1, a)$ from the channel and replace it with $f(d'_1, a)$ where d'_1 is a message of the attacker's own choosing. It can send $f(d'_1, a)$ after $f(d_1, a)$ is sent, but $f(d'_1, a)$ may not be accepted if the secret parameter is changed each time the message is sent. The enforcement of the policy on the sharing and updating of parameters is the job of the dialect, which will be discussed in Section 4.

Definition 1 (Lingo). *A* lingo *Λ is a 5-tuple $\Lambda = (D_1, D_2, A, f, g)$, where D_1, D_2 and A are sets called, respectively, the* input, output *and* parameter *sets, f, g are functions $f : D_1 \times A \to D_2$, $g : D_2 \times A \to D_1$ such that*[5] *$\forall d_1 \in D_1$, $\forall a \in A$, $g(f(d_1, a), a) = d_1$. We call a lingo* non-trivial *iff D_1, D_2 and A are non-empty sets. In what follows, all lingos considered will be non-trivial.*

Note that a lingo $\Lambda = (D_1, D_2, A, f, g)$ is just a three-sorted (Σ, E)-algebra, with sorts[6] *D_1, D_2 and A, function symbols $f : D_1 \times A \to D_2$ and $g : D_2 \times A \to D_1$, and E the single Σ-equation $g(f(d_1, a), a) = d_1$.*

[5] The equality $g(f(d_1, a), a) = d_1$ can be generalized to an equivalence $g(f(d, a), a) \equiv d$. The generalization of lingos to allow for such message equivalence is left for future work.

[6] Note the slight abuse of notation: in Σ, D_1, D_2 and A are uninterpreted sort *names*, whereas in a given lingo $\Lambda' = (D'_1, D'_2, A', f', g')$, such sorts are respectively interpreted as *sets* D'_1, D'_2 and A' and, likewise, the uninterpreted function *symbols* f and g in Σ are interpreted as actual *functions* f' and g' in a given lingo.

Note the asymmetry between f and g, in the lingo definition, since we do not have an equation of the form $\forall d_2 \in D_2$, $\forall a \in A$, $f(g(d_2, a), a) = d_2$. The reason for this asymmetry is that, given $a \in A$, the set $\{f(d_1, a) \mid d_1 \in D_1\}$ may be a *proper* subset of D_2. However, we show in [9] that the equation $f(g(d_2, a), a) = d_2$ does hold for any $d_2 \in \{f(d_1, a) \mid d_1 \in D_1\}$. Other results and proofs are included in [9]. Of course, for the special case of a lingo such that $\forall a \in A$, $D_2 = \{f(d_1, a) \mid d_1 \in D_1\}$, the equation $\forall d_2 \in D_2$, $\forall a \in A$, $f(g(d_2, a), a) = d_2$ will indeed hold. In particular, the set equality $D_2 = \{f(d_1, a) \mid d_1 \in D_1\}$ holds for all $a \in A$ in the following example.

Example 1 (The XOR{n} Lingo). Let $\Lambda_{xor}\{n\} = (\{0,1\}^n, \{0,1\}^n, \{0,1\}^n, \oplus, \oplus)$, with $_ \oplus _ : \{0,1\}^n \times \{0,1\}^n \to \{0,1\}^n$ the bitwise exclusive or operation.

Note that $\Lambda_{xor}\{n\}$ is *parametric* on $n \in \mathbb{N} \setminus \{0\}$. That is, for each choice of $n \geq 1$ we get a corresponding lingo. This is a common phenomenon: for many lingos, D_1, D_2 and A are not fixed sets, but *parametrized data types*, so that for each choice of the parameter we get a corresponding instance lingo. For a more detailed explanation of *parametrized data types*, see [9].

Example 2 (XOR-BSEQ Lingo for Bit-sequences). Yet another variation on the same theme is to define the $\Lambda_{xor.BSeq}$ lingo, whose elements are *bit sequences* of arbitrary length.

Note that $\{0,1\}^n$ is not the only possible parametrized data type on which a lingo based on the *xor* operation could be based. A different parametrized data type can have as elements the *finite subsets* of a parameter set D, and where the *xor* operation is interpreted as the *symmetric difference* of two finite subsets of D. A finite subset, say, $\{a, b, c\} \subseteq D$, with $a, b, c \in D$ different elements, can then be represented as the expression a *xor* b *xor* c. For D a finite set, there is an isomorphism between its power set $\mathcal{P}(D)$ with symmetric difference, and the function set $[D \to \{0,1\}]$ with pointwise *xor* of predicates $p, q \in [D \to \{0,1\}]$, i.e., p *xor* $q = \lambda d \in D.\ p(d)$ *xor* $q(d)$, which when $|D| = n$ is isomorphic to $\{0,1\}^n$ with pointwise *xor*.

Example 3 (Divide and Check (D&C) Lingo). Let $\Lambda_{D\&C} = (\mathbb{N}, \mathbb{N} \times \mathbb{N}, \mathbb{N}, f, g)$, given $\forall n, a, x, y \in \mathbb{N}$ then:

- $f(n, a) = (quot(n + (a+2), a+2), rem(n + (a+2), a+2))$
- $g((x, y), a) = (x \cdot (a+2)) + y - (a+2)$

where *quot* and *rem* denote the quotient and remainder functions on naturals.

The idea of the $\Lambda_{D\&C}$ lingo is quite simple. Given a parameter $a \in \mathbb{N}$, an input number n is transformed into a pair of numbers (x, y): the quotient x of $n+a+2$ by $a+2$ (this makes sure that $a+2 \geq 2$), and the remainder y of $n+a+2$ by $a+2$. The meaning of "divide" is obvious. The meaning of "check" reflects the fact that a receiver of a pair (x, y) who knows a can check $x = quot((x \cdot (a+2)) + y, a+2))$ and $y = rem((x \cdot (a+2)) + y, a+2))$, giving some assurance that the pair (x, y) was obtained from $n = (x \cdot (a+2)) + y - (a+2)$ and has not been tampered with.

This is an example of an *f-checkable* lingo (see Section 2.1 below), whereas no such check is possible for any of the isomorphic versions of the Λ_{xor} lingo (see again Section 2.1 below).

For a detailed explanation with Maude formal executable specifications of Examples 1, 2 and 3, we refer the reader to [9].

2.1 f-Checkable Lingos and the $\Lambda \mapsto \Lambda^\sharp$ Lingo Transformation

In this section we discuss lingos where the validity of the generated payloads can be checked by the dialect. We define a lingo transformation that substantially reduces the probability of an attacker successfully forging a message.

Definition 2 (f-Checkable Lingo). *A Lingo* $\Lambda = (D_1, D_2, A, f, g)$ *is called* f-checkable *iff* $\forall a \in A$, $\exists d_2 \in D_2$ *s.t.* $\nexists\, d_1 \in D_1$ *s.t.* $d_2 = f(d_1, a)$.

Above, Λ is understood, not as a fixed lingo, but as an equational theory specifying lingos. Therefore, the "f" in "f-Checkable Lingo" is an uninterpreted function symbol. However, once an actual interpretation for the theory Λ is given, and therefore for D_1, D_2, A, f and g, f becomes interpreted as the concrete function specified by that interpretation.

Note that the Λ_{xor} lingo of Example 1 is not f-checkable. We call a lingo $\Lambda = (D_1, D_2, A, f, g)$ *symmetric* iff (D_2, D_1, A, g, f) is also a lingo. Obviously, Λ_{xor} is symmetric. Any symmetric lingo is *not* f-checkable, since any $d_2 \in D_2$ satisfies the equation $f(g(d_2, a), a) = d_2$.

Example 4. The $\Lambda_{D\&C}$ lingo of Example 3 is f-checkable. Indeed, for each $a \in \mathbb{N}$, any (x, y) with $y \geq a + 2$ cannot be of the form $(x, y) = f(n, a)$ for any $n \in \mathbb{N}$. This is a cheap, easy check. The actual check that a d_1 exists, namely, the check $(x, y) = f(g((x, y), a), a)$ provided by [9] (where *proofs of all theorems are given*), was already explained when making explicit the meaning of "C" in $\Lambda_{D\&C}$.

The following theorem explains how to build an f-checkable lingo. The proof is in [9].

Theorem 1. *Let* $\Lambda = (D_1, D_2, A, f, g)$ *be a lingo, where we assume that* D_1, D_2 *and* A *are computable data-types and* $|D_1| \geq 2$. *Then,* $\Lambda^\sharp = (D_1, D_2 \times D_2, A \otimes A, f^\sharp, g^\sharp)$ *is an* f*-checkable lingo, where:*

- $A \otimes A = A \times A \setminus id_A$, *with* $id_A = \{(a, a) \in A^2 | a \in A\}$
- $f^\sharp(d_1, (a, a')) = (f(d_1, a), f(d_1, a'))$
- $g^\sharp((d_2, d_2'), (a, a')) = g(d_2, a)$.

Example 5 (f-checkable transformed version of $\Lambda_{xor}\{n\}$*).* The lingo transformation $\Lambda \mapsto \Lambda^\sharp$ maps $\Lambda_{xor}\{n\}$ in Example 1 to the f-checkable lingo $\Lambda^\sharp_{xor}\{n\} = (\{0,1\}^n, \{0,1\}^n \times \{0,1\}^n, \{0,1\}^n \otimes \{0,1\}^n, \oplus^\sharp, \oplus^\sharp)$.

2.2 Malleable Lingos

A cryptographic function is called *malleable* if an intruder, *without knowing the secret key*, can use one or more existing encrypted messages to generate another, related, message also encrypted with the same secret key. In a similar way, if a lingo Λ is what below we call *malleable*, an intruder can disrupt the communication between an honest sender *Alice* and an honest receiver *Bob* in a protocol $\mathcal{P}$ whose messages are modified by means of a lingo[7] Λ by producing a message *compliant* with a secret parameter $a \in A$ of Λ supposedly sent from *Alice* to *Bob* (who share the secret parameter a), but actually sent by the intruder.

Definition 3 (Malleable Lingo). *Let $\Lambda = (D_1, D_2, A, f, g)$ be a lingo, which has been formally specified as the initial algebra of an equational theory $(\Sigma_\Lambda, E_\Lambda)$. Λ is called* malleable *iff there exists a* recipe*, i.e., a Σ_Λ-term $t(x, y)$ of sort D_2 with free variables x, y of respective sorts D_2 and A, as well as a subset $A_0 \subseteq A$ such that $\forall d_1 \in D_1$, $\forall a \in A$, $\forall a' \in A_0$,*

1. *$f(d_1, a) \neq t(f(d_1, a), a')$, where, by convention, $t(f(d_1, a), a')$ abbreviates the substitution instance $t\{x \mapsto f(d_1, a), y \mapsto a'\}$*
2. *$\exists d'_1 \in D_1$ such that $t(f(d_1, a), a') = f(d'_1, a)$.*

Example 6. The lingo $\Lambda_{xor}\{n\}$ from Example 1 is *malleable*. The recipe $t(x, y)$ is $x \oplus y$, and $A_0 = \{0,1\}^n \setminus \{\overrightarrow{0}\}$. Then, for any $a' \in A_0$ we have, $t(f(d_1, a), a') = d_1 \oplus a \oplus a' \neq d_1 \oplus a$, which meets condition (1), since $a' \neq \overrightarrow{0}$ and $\oplus$ is a group addition; and it meets condition (2), since $t(f(d_1, a), a') = f(d_1 \oplus a', a)$. An attacker needs not know $d_1 \oplus a'$.

The notion of an f-checkable lingo of Section 2.1 provides a first line of defense against an intruder trying to generate a payload compliant with a secret parameter a. However, the example below shows that an f-checkable lingo may be malleable.

Example 7. The lingo $\Lambda^\sharp_{xor}\{n\}$ from Example 5 with $n \geq 2$ is malleable with recipe $t((x_1, x_2), (y_1, y_2)) = (x_1 \oplus y_1, x_2 \oplus y_1)$ and $A_0 = \{(y_1, y_2) \in \{0,1\}^n \otimes \{0,1\}^n \mid y_1 \neq \overrightarrow{0}\}$. Indeed, for any $d_1 \in \{0,1\}^n$, $(a_1, a_2) \in \{0,1\}^n \otimes \{0,1\}^n$, and $(a'_1, a'_2) \in A_0$, we have:

1. $t(f^\sharp(d_1, (a_1, a_2)), (a'_1, a'_2))) = (d_1 \oplus a_1 \oplus a'_1, d_1 \oplus a_2 \oplus a'_1)$ so that, by $a'_1 \neq \overrightarrow{0}$, we have $f^\sharp(d_1, (a_1, a_2)) \neq t(f^\sharp(d_1, (a_1, a_2)), (a'_1, a'_2)))$.
2. From $t(f^\sharp(d_1, (a_1, a_2)), (a'_1, a'_2))) = (d_1 \oplus a_1 \oplus a'_1, d_1 \oplus a_2 \oplus a'_1)$ it trivially follows that there is a d'_1, namely, $d'_1 = d_1 \oplus a'_1$, such that $f^\sharp(d'_1, (a_1, a_2)) = (d_1 \oplus a'_1 \oplus a_1, d_1 \oplus a'_1 \oplus a_2) = t(f^\sharp(d_1, (a_1, a_2)), (a'_1, a'_2)))$.

Therefore, $\Lambda^\sharp_{xor}\{n\}$ with $n \geq 2$ is malleable. An attacker needs not know d'_1, only needs to use $f^\sharp(d_1, (a_1, a_2))$, choose $(a, a') \in A_0$, and then use t.

[7] I.e., honest participants that use a *dialect* $\mathcal{D}_\Lambda(\mathcal{P})$ based on $\mathcal{P}$ and Λ to modify their messages (see §4).

Examples 6–7 suggest the following generalization of Theorem 1 above. The proof is in [9].

Theorem 2. *Let $\Lambda = (D_1, D_2, A, f, g)$ be a lingo which has been formally specified as the initial algebra of an equational theory $(\Sigma_\Lambda, E_\Lambda)$ and is such that: (i) $D_2 \subseteq D_1$, (ii) there is a subset $A_0 \subseteq A$ with at least two different elements a_0, a_1 and such that $\forall d_1 \in D_1$, $\forall a' \in A_0$, $f(d_1, a') \neq d_1$, and (iii) $\forall\ d_1 \in D_1$, $\forall a \in A$, $\forall a' \in A_0$, $f(f(d_1, a), a') = f(f(d_1, a'), a)$. Then both Λ and $\Lambda^\sharp$ are malleable.*

Remark 1. Note that we don't require a specification of the relation between the two plaintexts d_1 and d_1' (as is done in the informal definition given at the beginning of this section) except for both belonging to D_1. This still captures the way in which an attacker would take advantage of a malleable function but is more straightforward to reason about.

Remark 2. Note that the recipes are used with two arguments: the ciphertext $f(d_1, a)$ and a parameter a' in A_0. It is also possible to ignore $f(d_1, a)$, i.e., for some lingos the attacker may generate a random cyphertext that, due to malleablity, can be mistaken for a genuine message by the receiver. More generally, other notions of lingo malleability besides that in Def. 3 may be studied.

Remark 3. Malleability is in general undesirable, but some threats introduced by it are mitigated by the on-path attacker model. For example, the on-path attacker cannot substitute a message $f(d_1', a)$ for the message $f(d_1, a)$ from which it was derived. But there are also other cases, for example if we allow repeated parameters, or encounter a situation in which an attacker can send a random message which a receiver could mistake for some $f(d_1, a)$, as in Remark 2.

Finally, one would like to transform a possibly malleable lingo Λ into a non-malleable lingo Λ'. Developing general methods for proving that a lingo is non-malleable is a topic for future research. Note, however, that in cryptography non-malleability of a cryptosystem has been shown to be equivalent to authenticated encryption [2, 3]. This may or may not be the case for protocol dialects, which use a weaker attacker model. In [9], we discuss *authenticating lingos* and a transformation mapping a lingo to an authenticating one which we conjecture can be used to build non-malleable lingos out of malleable ones.

3 Lingo Compositions

Lingos can be composed in various ways to obtain new lingos. Lingo compositions provide modular, automated methods of obtaining new lingos from existing ones. Such compositions are often stronger, i.e., harder to compromise by an attacker, than the lingos so composed. We define here two such lingo composition operations, namely, *horizontal* and *functional* composition of lingos.

3.1 Horizontal Composition of Lingos

Given a finite family of lingos $\overrightarrow{\Lambda} = \{\Lambda_i\}_{1 \leq i \leq k}$, $k \geq 2$, all sharing the same input data type D_1, their *horizontal composition*, denoted $\bigoplus \overrightarrow{\Lambda}$ is intuitively their union, i.e., in $\bigoplus \overrightarrow{\Lambda}$ the input data type D_1 remains the same, D_2 is the union of the $D_{2.i}$, $1 \leq i \leq k$, and A is the disjoint union of the A_i, $1 \leq i \leq k$. $\bigoplus \overrightarrow{\Lambda}$ is a more complex lingo than each of its component lingos Λ_i, making it harder to compromise by an attacker, because it becomes a hydra with k heads: quite unpredictably, sometimes behaves like some Λ_i and sometimes like another Λ_j. Furthermore, it has a bigger parameter space, since $|A| = \Sigma_{1 \leq i \leq k} |A_i|$.

Definition 4 (Horizontal Composition). *Let $\overrightarrow{\Lambda}$ be a finite family of lingos $\overrightarrow{\Lambda} = \{\Lambda_i\}_{1 \leq i \leq k}$, $k \geq 2$, all having the same input data type D_1, i.e., $\Lambda_i = (D_1, D_{2.i}, A_i, f_i, g_i)$, $1 \leq i \leq k$. Let $\overrightarrow{d}_0 = (d_{0.1}, \ldots, d_{0.k})$ be a choice of default $D_{2.i}$-values, $d_{0.i} \in D_{2.i}$, $1 \leq i \leq k$. Then, the* horizontal composition *of the lingos $\overrightarrow{\Lambda}$ with default $D_{2.i}$-values $\overrightarrow{d}_0$ is the lingo:*

$$\bigoplus_{\overrightarrow{d_0}} \overrightarrow{\Lambda} = (D_1, \bigcup_{1 \leq i \leq k} D_{2.i}, \bigcup_{1 \leq i \leq k} A_i \times \{i\}, \oplus \overrightarrow{f}, \oplus \overrightarrow{g})$$

where, for each $d_1 \in D_1$, $d_2 \in \bigcup_{1 \leq i \leq k} D_{2.i}$, and $a_i \in A_i$, $1 \leq i \leq k$,

1. $\oplus \overrightarrow{f} = \lambda(d_1, (a_i, i)).f_i(d_1, a_i)$,
2. $\oplus \overrightarrow{g} = \lambda(d_2, (a_i, i))$. **if** $d_2 \in D_{2.i}$ **then** $g_i(d_i, a_i)$ **else** $g_i(d_{0.i}, a_i)$.

$\bigoplus_{\overrightarrow{d_0}} \overrightarrow{\Lambda}$ is indeed a lingo: $\oplus \overrightarrow{g}(\oplus \overrightarrow{f}(d_1, (a_i, i)), (a_i, i)) = g_i(f_i(d_1, a_i), a_i) = d_1$.

We have already pointed out that, in practice, two protocol participants using a lingo to first modify and then decode a payload d_1 that, say, Alice sends to Bob, agree on a secret parameter $a \in A$ by agreeing on a secret random number n, because both use a function $\mathit{param} : \mathbb{N} \to A$ to get $a = \mathit{param}(n)$. This then poses the practical problem of synthesizing a function $\oplus \overrightarrow{\mathit{param}} : \mathbb{N} \to \bigcup_{1 \leq i \leq k} A_i \times \{i\}$ for $\bigoplus_{\overrightarrow{d_0}} \overrightarrow{\Lambda}$ out of the family a functions $\{\mathit{param}_i : \mathbb{N} \to A_i\}_{1 \leq i \leq k}$ used in each of the lingos Λ_i. Furthermore, different lingos in the family $\overrightarrow{\Lambda}$ may have different degrees of strength against an adversary. This suggests favoring the choice of stronger lingos over that of weaker lingos in the family $\overrightarrow{\Lambda}$ to achieve a function $\oplus \overrightarrow{\mathit{param}}$ that improves the overall strength of $\bigoplus_{\overrightarrow{d_0}} \overrightarrow{\Lambda}$. This can be achieved by choosing a *bias vector* $\overrightarrow{\beta} = (\beta_1, \ldots, \beta_k) \in \mathbb{N}^k_{>0}$, so that, say, if lingo Λ_i is deemed to be stronger than lingo Λ_j, then the user chooses $\beta_i > \beta_j$. That is, $\overrightarrow{\beta}$ specifies a *biased* die with $k \geq 2$ faces, so that the die will show face j with probability $\frac{\beta_j}{\Sigma_{1 \leq i \leq k} \beta_i}$. Therefore, we can use a pseudo-random function $\mathit{throw}_{\overrightarrow{\beta}} : \mathbb{N} \to \{1, \ldots, k\}$ simulating a sequence of throws of a k-face die with bias $\overrightarrow{\beta}$ to get our desired function $\oplus \overrightarrow{\mathit{param}}$ as the function:

$$\oplus \overrightarrow{\mathit{param}}(n) = (\mathit{param}_{\mathit{throw}_{\overrightarrow{\beta}}(n)}(n), \mathit{throw}_{\overrightarrow{\beta}}(n)).$$

Example 8. To see how horizontal composition can strengthen lingos, consider the D&C lingo described in Example 3, in which $f(n,a) = (x,y)$ where $x = quot(n + (a+2), a+2)$ and $y = rem(n + (a+2), a+2)$. We note that any choice of $0 \leq y < a+2$ will pass the f-check, so the choice of $y = 0$ or 1 will always pass the f-check. To counter this, let *reverse*-D&C be the lingo with $f(n,a) = (y,x)$, where y and x are computed as in D&C. The attacker's best strategy in reverse-D&C is the opposite of that in D&C. Thus composing D&C and reverse-D&C horizontally with a bias vector $(.5, .5)$ means that the attacker strategy of choosing the first (respectively, second) element of the output to be 0 succeeds with probability 0.5 in any particular instance, as opposed to probability 1 for D&C by itself.

The following result follows easily from the definition of horizontal composition.

Lemma 1. *If each Λ_i in $\overrightarrow{\Lambda} = \{\Lambda_i\}_{1 \leq i \leq k}$, $k \geq 2$, is f-checkable, then $\bigoplus_{\overrightarrow{d_0}} \overrightarrow{\Lambda}$ is also f-checkable.*

Example 9 (Horizontal composition of XOR-BSeq and D&C Lingos). The lingos $\Lambda_{xor.BSeq}$ of Example 2 and $\Lambda_{D\&C}$ of Example 3 share the same D_1, namely, $\mathbb{N}$. Therefore, they do have a horizontal composition $\Lambda_{xor.BSeq} \oplus_{\overrightarrow{d}_0} \Lambda_{D\&C}$ for any choice of $\overrightarrow{d}_0$.

We refer the reader to [9] for details on the executable specification of horizontal composition in Maude.

3.2 Functional Composition of Lingos

Given two lingos Λ and Λ' such that the output data type D_2 of Λ coincides with the input data type of Λ', it is possible to define a new lingo $\Lambda \odot \Lambda'$ whose f and g functions are naturally the compositions of f and f' (resp. g' and g) in a suitable way. Moreover, Λ and Λ' can be chosen so that $\Lambda \odot \Lambda'$ is harder to compromise than either Λ or Λ'.

Definition 5 (Functional Composition). *Given lingos $\Lambda = (D_1, D_2, A, f, g)$ and $\Lambda' = (D_2, D_3, A', f', g')$, their* functional composition *is the lingo $\Lambda \odot \Lambda' = (D_1, D_3, A \times A', f \cdot f', g * g')$, where for each $d_1 \in D_1$, $d_3 \in D_3$, and $(a, a') \in A \times A'$,*

- $f \cdot f'(d_1, (a, a')) =_{def} f'(f(d_1, a), a')$,
- $g * g'(d_3, (a, a')) =_{def} g(g'(d_3, a'), a)$.

$(D_1, D_3, A \times A', f \cdot f', g * g')$ is indeed a lingo, since we have:

$$\begin{aligned} g * g'(f \cdot f'(d_1, (a, a')), (a, a')) &= g(g'(f'(f(d_1, a), a'), a'), a) \\ &= g(f(d_1, a), a) = d_1. \end{aligned}$$

Lemma 2. *Given lingos $\Lambda = (D_1, D_2, A, f, g)$ and $\Lambda' = (D_2, D_3, A', f', g')$, if Λ' is f-checkable, then $\Lambda \odot \Lambda'$ is also f-checkable.*

Example 10 (Functional composition of XOR-BSeq and D&C Lingos). The lingos $\Lambda_{xor.BSeq}$ of Example 2 and $\Lambda_{D\&C}$ of Example 3 are functionally composable as $\Lambda_{xor.BSeq} \odot \Lambda_{D\&C}$, because $\mathbb{N}$ is both the output data type of $\Lambda_{xor.BSeq}$ and the input data type of $\Lambda_{D\&C}$. Note that, by Lemma 2 above, and Theorem 1 above, $\Lambda_{xor.BSeq} \odot \Lambda_{D\&C}$ is f-checkable, in spite of $\Lambda_{xor.BSeq}$ not being so.

We refer the reader to [9] for details on the executable specification of functional composition in Maude.

4 Lingo Transformations and Dialects as Formal Patterns

We make the notion of a *formal pattern* as a transformation of rewrite theories explicit and explain how it applies to both lingos and dialects. Since dialects are parametric on the chosen lingo and the chosen protocol, we begin by explaining how protocols are formalized as generalized actor rewrite theories. With all these notions in hand we can explain in detail the *lingo formal pattern* and its rewriting logic semantics and give examples of different dialects for a given protocol based on the choice of various lingos, sometimes themselves compositions of simpler ones. Finally, we explain how both the lingo formal pattern, lingos and dialect formal patterns are specified as *formal executable specifications* in the Maude rewriting logic language [5], a theme further developed in [9].

4.1 Protocols as Generalized Actor Rewrite Theories

Actors [1] are a widely used model for distributed systems, in which concurrent objects communicate through asynchronous message passing. When an actor consumes a received message it can change its state, send new messages, and create new actors. Actor systems can be formally specified and executed as *actor rewrite theories* [17]. In *rewriting logic* [16] a concurrent system can be formally specified and executed as a *rewrite theory* $\mathcal{R} = (\Sigma, E, R)$, with (Σ, E) an equational theory with function symbols Σ and equations E that specifies the concurrent system's *states* as elements of the initial algebra (algebraic data type) $\mathbb{T}_{\Sigma/E}$ of (Σ, E), and where the system's *local concurrent transitions* are specified by *rewrite rules* in R of the form $u(x_1, \ldots, x_n) \rightarrow v(x_1, \ldots, x_n)$, where $u(x_1, \ldots, x_n)$ is a Σ-expression identifying those local state fragments that can make a local transition to a state fragment of the form $v(x_1, \ldots, x_n)$. Under natural assumptions, a rewrite theory $\mathcal{R} = (\Sigma, E, R)$ is *executable* by rewriting. Maude [5] is a declarative programming language whose programs, called *system modules*, are rewrite theories declared as `mod` (Σ, E, R) `endm`. Maude has a functional sublanguage of *functional modules*, which are equational theories declared as `fmod` (Σ, E) `endfm`, specifying an algebraic data type $\mathbb{T}_{\Sigma/E}$. For example, (more on this in Section 4.3) lingos are algebraic data types that can be naturally specified as functional modules in Maude.

Any *communication protocol* $\mathcal{P}$ can be formally specified and programmed as a *generalized actor rewrite theory* $\mathcal{P} = (\Sigma, E \cup B, R)$ [12]. The rewrite rules

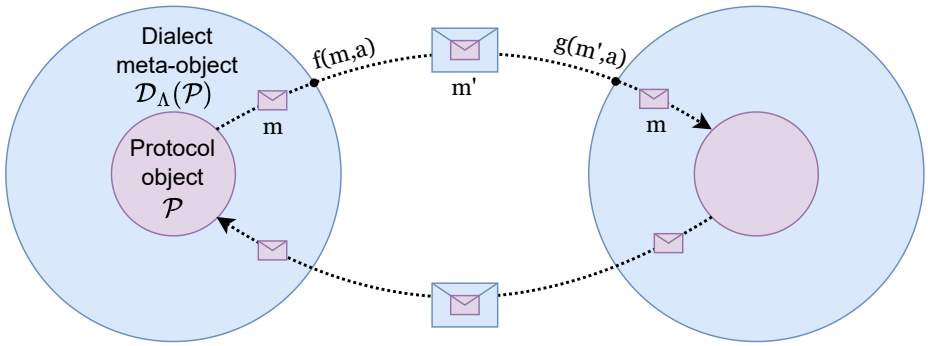

Fig. 1: Dialect meta-objects (enhanced from [8]).

of an actor rewrite theory are of the form:

$$(to\ B\ from\ A : M)\ <\ B : Cl \,|\, atts\ > \rightarrow <\ B : Cl \,|\, atts'\ >\quad msgs\ \ actors.$$

They specify how actor B of class Cl, upon receiving a message from actor A, can change its local state (the attribute-value pairs *atts*) to *atts'* and may generate several new messages *msgs* and several new actors *actors*. Any such rule specifies an *asynchronous* local concurrent transition in a distributed state (called a *configuration*) which is a multiset of objects and messages built up with a multiset union operation _ _ (juxtaposition) satisfying the axioms B of associativity, commutativity and empty multiset $\emptyset$ as unit element [17]. Generalized actor rewrite theories allow also rules of the form:

$$<\ B : Cl \,|\, atts\ > \rightarrow <\ B : Cl \,|\, atts'\ >\quad msgs\ \ objs.$$

describing *active actors* that can also change their state and generate new messages and actors without receiving a message.

Because of space limitations, we refer the reader to [9] for details on how lingos can be used as Formal Patterns.

4.2 The Dialect Transformation as a Formal Pattern

The *dialect transformation* can be formalized as a formal pattern of the form:

$$\mathcal{D} : GralActorRewThs \times PLingos \ni (\mathcal{P}, \Lambda_p) \mapsto \mathcal{D}_{\Lambda_p}(\mathcal{P}) \in GralActorRewThs$$

where $\mathcal{P}$ is a protocol, i.e., a rewrite theory in the class[8] *GralActorRewriteThs* of generalized actor rewrite theories, and $\mathcal{D}_{\Lambda_p}(\mathcal{P})$ is another protocol obtained from $\mathcal{P}$ by means of Λ_p, where Λ_p is a *lingo with parameter function* p in the class *PLingos*, that is, Λ_p adds to a lingo Λ a parameter function $p : \mathbb{N} \rightarrow A$.

The essential ideas about the dialect $\mathcal{D}_{\Lambda_p}(\mathcal{P})$ are summarized in Figure 1. Suppose that Alice and Bob are actors in the protocol $\mathcal{P}$ and Alice sends payload m to Bob in a message. What happens instead in $\mathcal{D}_{\Lambda_p}(\mathcal{P})$ is that *Alice and Bob*

[8] Hereafter we assume generalized actor systems whose actors do not create new actors.

$$
\begin{aligned}
&in\ :< O_1 : DC \mid in.buffer : M_{in}, atts' > \ (to\ O_1\ from\ O_2\ :\ P)\\
&\rightarrow\ < O_1 : DC \mid in.buffer : (M_{in} \cup (to\ O_1\ from\ O_2\ :\ P), atts' >\\
&out\ :< O_1 : DC \mid conf : (< O_1 : C \mid atts >\ (to\ O_2\ from\ O_1\ :\ P) \cup\ M),\\
&\qquad peer.counters : R,\ atts' >\\
&\rightarrow\ < O_1 : DC \mid conf : (< O_1 : C \mid atts >\ M \setminus (to\ O_2\ from\ O_1\ :\ P)),\\
&\qquad \text{update}(peer.counters : R, atts') >\\
&\quad (to\ O_2\ from\ O_1\ :\ f(P, param(R[O_2])))\\
&deliver\ :< O_1 : DC \mid conf : (< O_1 : C \mid atts > M),\ in.buffer : M_{in},\\
&\qquad peer.counters : R,\ atts' >\\
&\rightarrow\ < O_1 : DC \mid conf : (< O_1 : C \mid atts > M \cup \{g(M_{selected}, \text{param}(R[O_2]))\},\\
&\qquad in.buffer : (M_{in} \setminus M_{selected}),\\
&\qquad update(peer.counters : R, atts') >\\
&\quad if\ size(M_{in}) \geq egressArity\ \wedge M_{selected} := take(egressArity, M_{in}) \wedge\\
&\quad fromOidTag(M_{selected}, R) = true\ .
\end{aligned}
$$

Fig. 2: Meta-Actor Rewrite Rules.

behave as before, that is, their original protocol $\mathcal{P}$ remains the same. However, unbeknownst to Alice and Bob, their communication is now *mediated* by their corresponding *meta-actors* (also named Alice, resp. Bob) inside which they are now *wrapped*. Alice's meta-actor *transforms* the original payload m into $f(m, a)$, and Bob's meta-actor *decodes* $f(m, a)$ into $g(f(m, a), a) = m$ using Λ_p and the *shared secret parameter* a. Of course, if Bob replies to Alice with another message, the roles are exchanged: now Bob's meta-actor will transform the new payload m' to $f(m', a')$ and Alice's meta-actor with get back m' because they now share a *new* secret parameter a'. Since parameters change all the time, dialects become *moving targets* for an attacker.

But what is a *meta-actor*? In rewriting logic a meta-actor of class *Dialect* will have the general form: $< o : Dialect \mid conf :< o : C \mid atts >, datts >$. That is, one of the attributes in the state of the meta-actor is called *conf* and consists of a *configuration* of objects and messages with a *single object*, namely, $< o : C \mid atts >$ and, at various times, either an incoming message addressed to o, or outgoing messages sent by o to other actors.

datts denotes the rest of the dialect meta-actor's state (i.e., it remaining attribute-value pairs). What are they? Besides the object attribute *conf*, *Dialect* meta-actors have the following additional attributes: (i) *in.buffer*: a buffer of incoming messages to which the lingo's decoding function g has not yet been applied; and (ii) *peer.counters*: a map from actor names to natural numbers that serves as a counter for the number of messages exchanged. It is also used to generate (using the parameter function p) the new secret parameters used by the lingo's f and g functions for communicating with other actors.

The *behavior* of a *Dialect* meta-actor is specified by the rewrite rules in Figure 2. These can be summarized as follows: (i) rule *in* collects incoming messages for the wrapped actor in the *in.buffer* attribute; (ii) rule *out* processes messages produced by the wrapped protocol, sending the transformed messages into the network, by applying the lingo's f function with the corresponding parameter; and (iii) rule *deliver* processes messages stored in the buffer, by applying the lingo's g function with the proper parameter, delivering the original version of the message to the protocol. Rules *out* and *deliver* apply f and g, respectively, and supply the parameter a by calling the lingo's param function, which computes a parameter $a \in A$ from a number $n \in \mathbb{N}$. The dialect meta-object keeps parameters changing by supplying to the param function the current message number from the map in *peer.counter*. This turns dialects into moving targets.

Example 11 (Transforming protocols by means of Protocol Dialects). Let us leverage the various lingos introduced in previous sections to incrementally construct more sophisticated, and possibly more secure, protocols through the use of dialects. We use as core protocol a parametric version of MQTT, denoted $Mqtt\{D\}$ and explained in [9], where D represents the data-type of the message payloads (e.g., bit-sequences of $n \in \mathbb{N}$). Furthermore, we denote any given lingo as $\Lambda_{id}\{D\}$, where id identifies the lingo and D indicates the input data type. For instance, $\Lambda_{D\&C}\{BitSEQ\{N\}\}$ refers to the divide and check lingo over bit-sequences of length $n \in \mathbb{N}$.

As a first transformation, we can use the lingo presented in Example 1. To do so, we can instantiate a dialect $\mathcal{D}_{\Lambda_p}(\mathcal{P})$ over MQTT, using the xor lingo over a payload consisting of 1 byte, i.e. a bit-sequence (or vector) of length 8. The resulting protocol is denoted by $\mathcal{D}_{\Lambda_{XOR}\{BitSEQ\{8\}\}}(Mqtt\{BitSEQ\{8\}\})$.

It is important to note that we can, in a very intuitive way, *upgrade* or adapt the new protocol to meet specific needs. For instance, we can increase the parameter size by just increasing the input parameter. This leads to a new protocol such as $\mathcal{D}_{\Lambda_{XOR}\{BitSEQ\{256\}\}}(Mqtt\{BitSEQ\{256\}\})$, which operates over 32-byte payloads.

A similar approach can be applied to both parameters of a Dialect: the lingo and the underlying protocol. For instance, consider replacing the XOR{N} lingo for the XOR-BSEQ lingo introduced in Example 2. These kind of substitutions are valid provided that both the input data type of the lingo and protocol coincide. For this purpose, we can instantiate the protocol's data type to be the natural numbers, that is bit sequences of arbitrary length. Accordingly, the resulting protocol can be expressed as $\mathcal{D}_{\Lambda_{xor.BSeq}}(Mqtt\{Nat\})$. Furthermore, we can change the lingo to the divide and check lingo given in Example 3 in a straightforward way since both XOR-BSEQ and D&C share input data types. The resulting protocol is $\mathcal{D}_{\Lambda_{D\&C}}(Mqtt\{Nat\})$.

Furthermore, we can use lingo transformations to construct more sophisticated dialects. For example, one may horizontally compose XOR-BSEQ with D&C, as illustrated in Example 9. Then, we can use this new lingo to yield a new protocol defined as $\mathcal{D}_{\Lambda_{xor.BSeq}\oplus_{\vec{d}_0}\Lambda_{D\&C}}(Mqtt\{Nat\})$. Another protocol can be obtained by using the functional composition from Example 10. The resulting

protocol would be $\mathcal{D}_{\Lambda_{xor.BSeq} \odot \Lambda_{D\&C}}(Mqtt\{Nat\})$. For a more detailed description of all these instantiations, we refer the reader to [9].

Last but not least, this work provides a substantial extension and simplification of the previous theory of dialects in [8] that includes, not only new lingo properties and formal patterns, but the *replacement* of the former *horizontal*, resp. *vertical*, dialect composition operations in [8] by the simpler and more efficient notions of *horizontal*, resp. *functional*, lingo composition operations.

4.3 Lingos, Dialects, and their Formal Patterns in Maude

As shown in [9] and Section 4.2, dialects can be formalized as equational theories with initial semantics, and dialect transformations as formal patterns composing or transforming them. Likewise, we have seen that a dialect $\mathcal{D}_{\Lambda_p}(\mathcal{P})$ is a formal pattern transforming a generalized actor rewrite theory $\mathcal{P}$ into another generalized actor rewrite theory. Since Maude programs are exactly either equational theories with initial semantics, or rewrite theories, both lingos and dialects, as well as their transformations, can be made executable in Maude. In fact, all the theoretical concepts and examples presented in this paper have been formally specified in Maude at the same time that their theoretical properties were proved. But the interest of these formal specifications is not just theoretical. On the one hand, the D-transformation proposed in [13] provides a formal pattern to automatically transform generalized actor rewrite theories into distributed implementations; on the other hand, dialects formalized in Maude can be formally analyzed by various model checking verification techniques, both in Maude itself [5] and, for probabilistic properties, in the QMaude tool [23].

5 Concluding Remarks

Dialects are a resource-efficient approach as a first line of defense against outside attackers. Lingos are invertible message transformations used by dialects to thwart attackers from maliciously disrupting communication. In this paper we propose new kinds of lingos and make them stronger through lingo transformations, including composition operations, all of them new. These lingo transformations and composition operations are *formal patterns* with desirable security properties. We also propose a refined and simpler definition for dialects by replacing two former dialect composition operations in [8] by considerably simpler and more efficient lingo composition operations. All these concepts and transformations have been formally specified and made executable in Maude.

Our next step is to explore lingos in action as they would be used by dialects in the face of an on-path attacker. We have been developing a formal intruder model that, when combined with the dialect specifications provided in this paper, can be used for statistical and probabilistic model checking of dialects. This model provides the capacity to specify the probability that the intruder correctly guesses secret parameters. For this we are using QMaude [23], which supports probabilistic and statistical model checking analysis of Maude specifications.

Acknowledgements We cordially thank Daniel Galán for his important contributions to the early development of these ideas in [8]. We also thank Zachary J Flores for his comments. S. Escobar and V. García have been partially supported by the grant CIPROM/2022/6 funded by Generalitat Valenciana and by the grant PID2024-162030OB-100 funded by MCIN/AEI/10.13039/501100011033 and ERDF A way of making Europe.

References

1. Agha, G.: Actors: A Model of Concurrent Computation in Distributed Systems. MIT Press, Cambridge, MA, USA (1986)
2. Bellare, M., Desai, A., Pointcheval, D., Rogaway, P.: Relations among notions of security for public-key encryption schemes. In: Annual International Cryptology Conference. pp. 26–45. Springer (1998)
3. Bellare, M., Namprempre, C.: Authenticated encryption: Relations among notions and analysis of the generic composition paradigm. Journal of cryptology **21**(4), 469–491 (2008)
4. Chadha, R., Gunter, C.A., Meseguer, J., Shankesi, R., Viswanathan, M.: Modular preservation of safety properties by cookie-based dos-protection wrappers. In: International Conference on Formal Methods for Open Object-Based Distributed Systems. pp. 39–58. Springer (2008)
5. Clavel, M., Durán, F., Eker, S., Lincoln, P., Oliet, N.M., Meseguer, J., Talcott, C.: All About Maude - A High-Performance Logical Framework: How to Specify, Program, and Verify Systems in Rewriting Logic. Lecture Notes in Computer Science, Springer (July 2007). `https://doi.org/10.1007/978-3-540-71999-1`, `http://dx.doi.org/http://dx.doi.org/10.1007/978-3-540-71999-1`
6. Durán, F., Eker, S., Escobar, S., Martí-Oliet, N., Meseguer, J., Rubio, R., Talcott, C.: Programming open distributed systems in maude. In: Proceedings of the 26th International Symposium on Principles and Practice of Declarative Programming. pp. 1–12 (2024)
7. Eckhardt, J., Mühlbauer, T., AlTurki, M., Meseguer, J., Wirsing, M.: Stable availability under denial of service attacks through formal patterns. In: International Conference on Fundamental Approaches to Software Engineering. pp. 78–93. Springer (2012)
8. Galán, D., García, V., Escobar, S., Meadows, C., Meseguer, J.: Protocol dialects as formal patterns. In: European Symposium on Research in Computer Security. pp. 42–61. Springer (2023)
9. García, V., Escobar, S., Meadows, C., Meseguer, J.: A theory of composable lingos for protocol dialects. Tech. rep. (2026), `https://doi.org/10.48550/arXiv.2603.19908`
10. Gogineni, K., Mei, Y., Venkataramani, G., Lan, T.: Can you speak my dialect?: A Framework for Server Authentication using Communication Protocol Dialects. arXiv preprint arXiv:2202.00500 (2022). `https://doi.org/10.48550/ARXIV.2202.00500`, `https://arxiv.org/abs/2202.00500`, publisher: arXiv Version Number: 1
11. Gogineni, K., Mei, Y., Venkataramani, G., Lan, T.: Verify-pro: A framework for server authentication using communication protocol dialects. In: MILCOM 2022-2022 IEEE Military Communications Conference (MILCOM). pp. 450–457. IEEE (2022)

12. Liu, S., Meseguer, J., Ölveczky, P.C., Zhang, M., Basin, D.A.: Bridging the semantic gap between qualitative and quantitative models of distributed systems. Proc. ACM Program. Lang. **6**(OOPSLA2), 315–344 (2022)
13. Liu, S., Sandur, A., Meseguer, J., Ölveczky, P.C., Wang, Q.: Generating correct-by-construction distributed implementations from formal Maude designs. In: Proc. NASA Formal Methods: 12th International Symposium, NFM 2020,. Lecture Notes in Computer Science, vol. 12229, pp. 22–40. Springer (2020)
14. Lukaszewski, D., Xie, G.G.: Towards software defined layer 4.5 customization. In: 2022 IEEE 8th International Conference on Network Softwarization (NetSoft). pp. 330–338. IEEE (2022)
15. Mei, Y., Gogineni, K., Lan, T., Venkataramani, G.: MPD: moving target defense through communication protocol dialects. In: García-Alfaro, J., Li, S., Poovendran, R., Debar, H., Yung, M. (eds.) Security and Privacy in Communication Networks - 17th EAI International Conference, SecureComm 2021, Virtual Event, September 6-9, 2021, Proceedings, Part I. Lecture Notes of the Institute for Computer Sciences, Social Informatics and Telecommunications Engineering, vol. 398, pp. 100–119. Springer (2021). https://doi.org/10.1007/978-3-030-90019-9_6
16. Meseguer, J.: Conditional rewriting logic as a united model of concurrency. Theor. Comput. Sci. **96**(1), 73–155 (1992). https://doi.org/10.1016/0304-3975(92)90182-F, https://doi.org/10.1016/0304-3975(92)90182-F
17. Meseguer, J.: A Logical Theory of Concurrent Objects and its realization in the Maude Language. In: Agha, G., Wegner, P., Yonezawa, A. (eds.) Research Directions in Concurrent Object-Oriented Programming, pp. 314–390. MIT Press (1993)
18. Meseguer, J.: Taming distributed system complexity through formal patterns. Sci. Comput. Program. **83**, 3–34 (2014). https://doi.org/10.1016/j.scico.2013.07.004, https://doi.org/10.1016/j.scico.2013.07.004
19. Meseguer, J.: Building correct-by-construction systems with formal patterns. In: Madeira, A., Martins, M.A. (eds.) Recent Trends in Algebraic Development Techniques - 26th IFIP WG 1.3 International Workshop, WADT 2022, Aveiro, Portugal, June 28-30, 2022, Revised Selected Papers. Lecture Notes in Computer Science, vol. 13710, pp. 3–24. Springer (2022)
20. Meseguer, J.: Capturing system designs with formal executable specifications. In: Boronat, A., Fraser, G. (eds.) Fundamental Approaches to Software Engineering - 28th International Conference, FASE 2025. Lecture Notes in Computer Science, vol. 15693, pp. 1–32. Springer (2025)
21. Nigam, V., Talcott, C.: Automated construction of security integrity wrappers for industry 4.0 applications. Journal of Logical and Algebraic Methods in Programming **126**, 100745 (2022)
22. Ren, T., Williams, R., Ganguly, S., De Carli, L., Lu, L.: Breaking embedded software homogeneity with protocol mutations. In: Security and Privacy in Communication Networks: 18th EAI International Conference, SecureComm 2022, Virtual Event, October 2022, Proceedings. pp. 770–790. Springer (2023)
23. Rubio, R., Martí-Oliet, N., Pita, I., Verdejo, A.: Qmaude: quantitative specification and verification in rewriting logic. In: International Symposium on Formal Methods. pp. 240–259. Springer (2023)
24. Sjoholmsierchio, M.: Software-Defined Networks: Protocol Dialects. Master's thesis, Naval Postgraduate School, Monterey, California, USA (Dec 2019), http://hdl.handle.net/10945/64066
25. Sun, M., Meseguer, J., Sha, L.: A formal pattern architecture for safe medical systems. In: International Workshop on Rewriting Logic and its Applications. pp. 157–173. Springer (2010)

26. Talcott, C.: Dialects for coap-like messaging protocols. arXiv preprint arXiv:2405.13295 (2024)